MIT ACADEMY OF ENGINEERING

INTERNAL ASSESSMENT · IA 2

Containerization & Kubernetes Deployment Challenge

Packaging and orchestrating the Student Management Portal with Docker and Kubernetes

STUDENT	Sahas Nagar
PROGRAMME	B.Tech Computer Engineering
INSTITUTE	MIT Academy of Engineering (MITAOE), Alandi, Pune, Maharashtra
ACADEMIC YEAR	2026–2027
SEMESTER	As applicable to the course
SUBJECT	Cloud Native DevOps
COURSE CODE	2310423T26-27
ASSIGNMENT TYPE	Internal Assessment Activities

Technical implementation report · Prepared for academic assessment

Table of contents

1. Introduction
2. Objectives
3. Application overview
4. Containerization concepts
5. Docker architecture
6. Dockerfile design
7. Image build and container execution
8. Kubernetes architecture
9. Deployment manifest
10. Kubernetes Service and exposure
11. Deployment procedure and verification
12. Data persistence and replica behavior
13. Execution and verification results
14. Browser-rendered registration view
15. Results and observations
16. Conclusion
Execution evidence and scope
17. Complete implementation source
References

1. Introduction

Containerization packages an application and its runtime dependencies into an image that can be run consistently across compatible environments. Kubernetes orchestrates containers by scheduling Pods, maintaining desired replica counts, exposing services, and checking application health. This assignment extends the same Student Management Portal used in IA 1 into a Docker image and Kubernetes Deployment design.

2. Objectives

- Explain Docker architecture, images, containers, and container lifecycle.
- Prepare a small production-oriented Dockerfile and .dockerignore for the portal.
- Define a Kubernetes Deployment with two replicas, resource requests/limits, and health probes.
- Expose the application through a NodePort Service and describe verification commands.
- Record only the application tests actually run; do not claim Docker or Kubernetes execution where tools are unavailable.

3. Application overview

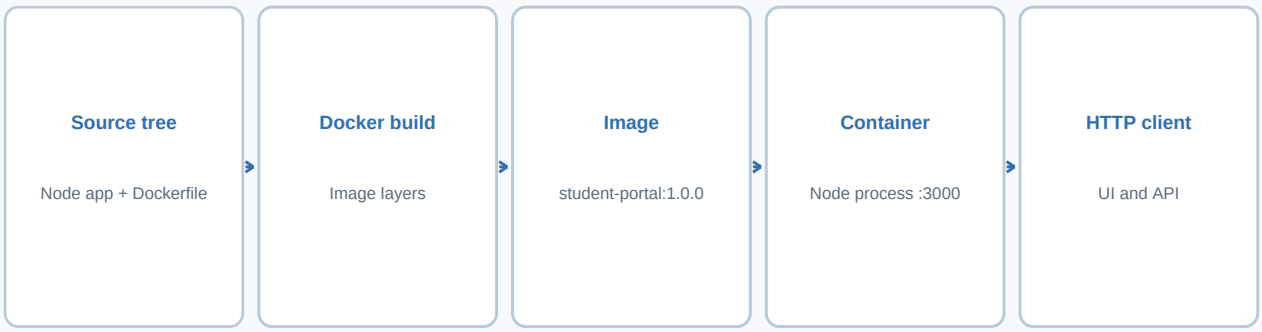
The portal is a dependency-free Node.js application. It serves a responsive dashboard at `/`, lists students at `/api/students`, registers students with POST requests, and exposes `/health`. The JSON file is stored alongside the application for this compact demonstration.

4. Containerization concepts

A container image is a read-only template composed of filesystem layers and metadata. A container is a running instance of that image with an isolated process namespace and resource controls supplied by the container runtime. The image should contain only what is needed at runtime, run as a non-root user where possible, expose the service port, and provide a health check.

5. Docker architecture

Figure 2.1 — Docker build and run flow



The Docker client sends build and run requests to the Docker Engine. The engine builds the image from the Dockerfile and starts containers from the resulting image. The application listens on port 3000 inside the container; publishing a host port makes the service reachable from the host.

6. Dockerfile design

The Dockerfile uses the official Node.js 22 Alpine image, sets production environment variables, creates a working directory, copies only runtime files with ownership assigned to the unprivileged `node` user, exposes port 3000, and defines a health check that requests `/health`. The container starts with the Node server process. The test directory is excluded from the image because tests run in the CI workspace rather than in the production image.

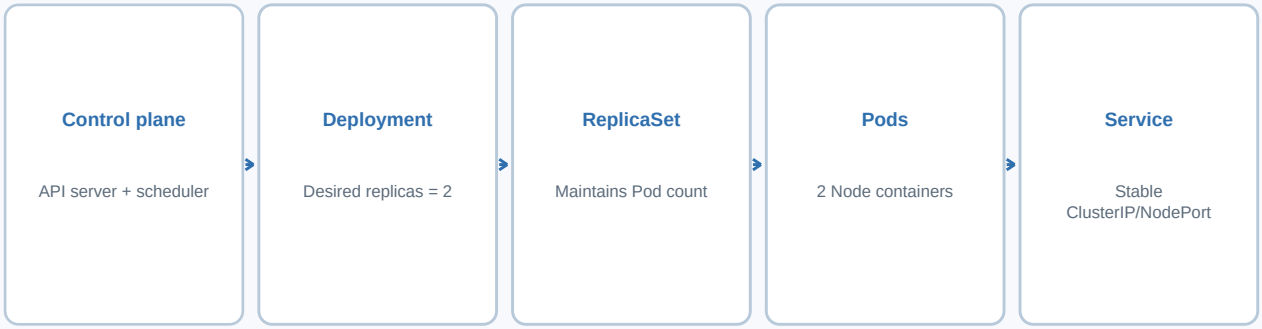
7. Image build and container execution

```
cd IA-2-Kubernetes/app
docker build -t student-portal:1.0.0 .
docker images student-portal
docker run --rm -d --name student-portal -p 3000:3000 student-portal:1.0.0
docker ps
curl --fail http://127.0.0.1:3000/health
docker logs student-portal
docker stop student-portal
```

The .dockerignore file excludes Git metadata, local node_modules, logs, test sources, and screenshots. This reduces build context size and prevents unrelated development files from entering the image.

8. Kubernetes architecture

Figure 2.2 — Kubernetes workload architecture



A Deployment describes the desired number of replicas and a Pod template. The Deployment controller creates or updates a ReplicaSet; the ReplicaSet maintains the requested Pod count. Each Pod runs the application container. A Service selects Pods by labels and provides a stable network endpoint even when individual Pods are replaced.

9. Deployment manifest

The Deployment declares two replicas and labels the workload `app: student-portal`. The Pod template uses the same label so the Service selector can target both Pods. The container exposes named port `http` on 3000. CPU and memory requests reserve scheduling capacity; limits cap resource consumption. Readiness checks gate traffic until the HTTP health endpoint succeeds. Liveness checks restart a container if the process stops responding.

10. Kubernetes Service and exposure

The Service uses type NodePort, selects the two application Pods, exposes port 80, and forwards to the named container port `http`. NodePort 30080 is within Kubernetes' default NodePort range. In Minikube, the reachable address may be obtained using `minikube service student-portal --url`; on Docker Desktop Kubernetes, the NodePort is commonly reachable through localhost.

11. Deployment procedure and verification

```
# Build into the image store used by the local cluster.
# For Minikube, use its Docker daemon before building:
eval $(minikube docker-env)
docker build -t student-portal:1.0.0 IA-2-Kubernetes/app

kubectl apply -f IA-2-Kubernetes/k8s/deployment.yaml
kubectl rollout status deployment/student-portal
kubectl get deployments
kubectl get replicasets
kubectl get pods -l app=student-portal
kubectl get services
kubectl get endpoints student-portal
kubectl describe deployment student-portal
curl --fail http://127.0.0.1:30080/health
```

The local image must be available to the Kubernetes node runtime. A host-built image is not automatically visible to every Minikube or Docker Desktop cluster configuration. For Minikube, build using its Docker environment or explicitly load the image; for Docker Desktop, ensure the cluster shares the local image cache. The manifest uses `IfNotPresent` so the node can use the locally loaded tag.

12. Data persistence and replica behavior

The current application writes to a JSON file inside its container filesystem. With two replicas, each Pod has its own writable layer. A registration handled by one Pod will not automatically appear in the other Pod, and data is lost when a Pod is replaced. This is a deliberate limitation of the lightweight assignment implementation, not a production-ready persistence strategy. A production design should use a shared database (for example PostgreSQL) with migrations, backups, credentials managed through Secrets, and suitable network policies.

13. Execution and verification results

Verified: the Node.js app and HTTP API ran in the available environment; health returned 200; student registration and listing behaved as expected; two automated Node tests passed. A browser-rendered UI image is included below.

Not verified: Docker Engine, Docker CLI, kubectl, and Minikube were not installed in the execution environment. Package downloads were blocked by DNS/network restrictions. No Docker image was built, no container was started, no Kubernetes Deployment or Service was applied, and no Pod/ReplicaSet/Endpoint status is claimed.

14. Browser-rendered registration view

This Chromium-rendered image shows the portal markup with a registration-success state and three sample records. It is a visual UI render, not a live container or Kubernetes browser session. The underlying registration API was tested independently against the running Node.js service.

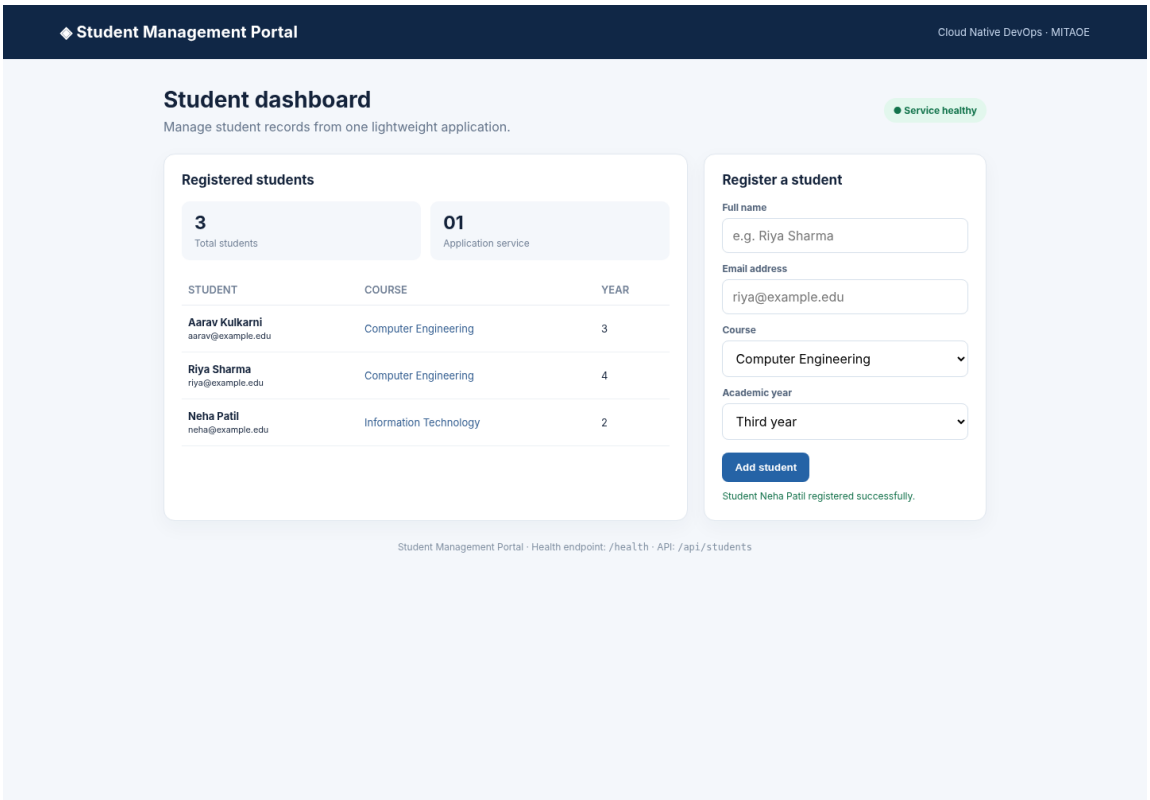


Figure 2.3 — Student registration interface render; Docker and Kubernetes execution were not available.

15. Results and observations

- The Dockerfile is small and uses an unprivileged runtime user.
- The Docker health check and Kubernetes readiness/liveness probes use the same HTTP health contract.
- The Deployment and Service selectors match the Pod template labels.
- Two replicas improve process availability but do not solve data consistency for a file-backed store.
- Container build and cluster deployment remain pending execution on a machine with Docker and a local Kubernetes cluster.

16. Conclusion

The Student Management Portal has a complete Dockerfile, .dockerignore, two-replica Kubernetes Deployment, and NodePort Service definition. The application itself is verified through HTTP tests, but the Docker and Kubernetes lifecycle was not run in the available environment. The report preserves this distinction and supplies the exact commands and manifests needed for a real local deployment.

Execution evidence and scope

This report separates verified execution from implementation artifacts that could not be run in the available sandbox. The execution host was Debian GNU/Linux 13 with Node.js v22.16.0, npm 10.9.2, Nginx 1.26.3, Git 2.47.3 and OpenJDK 21. It was not the student's Windows 11/WSL2 laptop. External package downloads were blocked by DNS/network restrictions.

Evidence integrity: no fabricated terminal output, Ansible recap, Docker build, Kubernetes pod status, Jenkins stage result, Terraform apply, or GitHub connection is presented. The three images in these reports are browser-rendered interface/health views generated in Chromium from the portal markup; they are not screenshots of a live interactive desktop browser session. API and reverse-proxy checks are separately recorded from real HTTP requests.

- **Executed:** Node.js application and API checks; two automated Node tests; Chromium-rendered UI preview.
- **Not executed:** docker build, docker images, docker ps, container health check, Kubernetes cluster startup, kubectl apply, two Ready replicas, Service endpoints, or NodePort browser access.
- **Operational caveat:** per-Pod JSON data is not shared; a database is required for consistent multi-replica registration.

No PRN, roll number, faculty name, signature, or marks have been invented. The semester is left as “As applicable to the course” because no semester value was supplied.

17. Complete implementation source

17.1 package.json

```
{
  "name": "student-management-portal",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "node server.js",
    "test": "node --test"
  }
}
```

17.2 server.js

```
const http = require('http');
const fs = require('fs');
const path = require('path');
const PORT = Number(process.env.PORT || 3000);
const DATA = process.env.DATA_PATH || path.join(__dirname, 'students.json');
const html = fs.readFileSync(path.join(__dirname, 'index.html'));
function readStudents() {
  try { return JSON.parse(fs.readFileSync(DATA, 'utf8')); }
  catch { return [{id:1,name:'Aarav Kulkarni',email:'aarav@example.edu',course:'Computer Engineering',year:3}];
}
}
function writeStudents(rows) { fs.writeFileSync(DATA, JSON.stringify(rows, null, 2)); }
function send(res, status, body, type='application/json') {
  res.writeHead(status, {'Content-Type': type, 'Cache-Control':'no-store'});
  res.end(type.includes('json') ? JSON.stringify(body) : body);
}
const server = http.createServer((req,res) => {
  const url = new URL(req.url, `http://${req.headers.host || 'localhost'}`);
  if (req.method === 'GET' && url.pathname === '/health') return
send(res,200,{status:'ok',service:'student-management-portal'});
  if (req.method === 'GET' && url.pathname === '/api/students') return send(res,200,readStudents());
  if (req.method === 'POST' && url.pathname === '/api/students') {
    let raw=''; req.on('data', c => raw += c);
    req.on('end', () => {
      try {
        const data=JSON.parse(raw);
        if (!data.name || !data.email || !data.course) return send(res,400,{error:'name, email and course are
required'});
        const rows=readStudents();
        const
student={id:rows.reduce((m,x)=>Math.max(m,x.id),0)+1,name:String(data.name).trim(),email:String(data.email).trim
(),course:String(data.course).trim(),year:Number(data.year||1)};
        rows.push(student); writeStudents(rows); send(res,201,student);
      } catch { send(res,400,{error:'invalid JSON'}); }
    }); return;
  }
  if (req.method === 'GET' && url.pathname === '/') return send(res,200,html.toString(),'text/html;
charset=utf-8');
  return send(res,404,{error:'not found'});
});
server.listen(PORT,'0.0.0.0',()=>console.log(`Student Management Portal listening on
http://127.0.0.1:${PORT}`));
```

17.3 index.html

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Student Management Portal</title>
<style>
:root{font-family:Inter,Segoe UI,Arial,sans-serif;color:#14233b;background:#f4f7fb}
*{box-sizing:border-box}body{margin:0}.top{background:#102746;color:white;padding:22px
max(5vw,24px);display:flex;align-items:center;justify-content:space-between}
.brand{font-size:20px;font-weight:750;letter-spacing:.2px}.tag{color:#c9d9ef;font-size:13px}
main{max-width:1080px;margin:34px auto;padding:0
22px}.hero{display:flex;justify-content:space-between;gap:20px;align-items:center;margin-bottom:24px}
h1{font-size:28px;margin:0 0
7px}.muted{color:#65758b;margin:0}.pill{background:#e2f7ed;color:#11734b;padding:8px
12px;border-radius:30px;font-size:12px;font-weight:700}
.grid{display:grid;grid-template-columns:1.3fr .7fr;gap:20px}.panel{background:white;border:1px solid
#e1e8f0;border-radius:14px;padding:22px;box-shadow:0 8px 24px #1c35500a}
.panel h2{font-size:17px;margin:0 0
17px}.stats{display:flex;gap:12px;margin-bottom:18px}.stat{background:#f4f7fb;border-radius:10px;padding:13px
16px;flex:1}.stat b{font-size:23px;display:block}.stat span{font-size:12px;color:#718096}
table{width:100%;border-collapse:collapse;font-size:13px}th{text-align:left;color:#718096;font-weight:600;paddin
g:11px 8px;border-bottom:1px solid #e8edf3}td{padding:13px 8px;border-bottom:1px solid
#edf1f5}.course{color:#335e91}
label{display:block;font-size:12px;font-weight:650;color:#52637a;margin:12px 0
6px}input,select{width:100%;padding:11px 12px;border:1px solid
#d6e0eb;border-radius:8px;font:inherit;background:#fff}
button{background:#2563a6;color:#fff;border:0;border-radius:8px;padding:11px
16px;font-weight:650;cursor:pointer;margin-top:16px}button:hover{background:#194e86}
#notice{font-size:12px;color:#13724b;margin-top:10px;min-height:16px}.footer{font-size:12px;color:#8190a2;text-a
lign:center;margin:25px}
@media(max-width:760px){.grid{grid-template-columns:1fr}.hero{align-items:flex-start;flex-direction:column}}
</style></head><body>
<header class="top"><div class="brand">■ Student Management Portal</div><div class="tag">Cloud Native DevOps ·
MITAOE</div></header>
<main><section class="hero"><div><h1>Student dashboard</h1><p class="muted">Manage student records from one
lightweight application.</p></div><span class="pill" id="health">● Checking service</span></section>
<div class="grid"><section class="panel"><h2>Registered students</h2><div class="stats"><div class="stat"><b
id="count">0</b><span>Total students</span></div><div class="stat"><b>01</b><span>Application
service</span></div></div>
<table><thead><tr><th>STUDENT</th><th>COURSE</th><th>YEAR</th></tr></thead><tbody
id="rows"></tbody></table></section>
<section class="panel"><h2>Register a student</h2><form id="form"><label for="name">Full name</label><input
id="name" required placeholder="e.g. Riya Sharma"><label for="email">Email address</label><input id="email"
type="email" required placeholder="riya@example.edu"><label for="course">Course</label><select
id="course"><option>Computer Engineering</option><option>Information Technology</option><option>Mechanical
Engineering</option><option>Electronics Engineering</option></select><label for="year">Academic
year</label><select id="year"><option value="1">First year</option><option value="2">Second
year</option><option value="3" selected>Third year</option><option value="4">Final
year</option></select><button type="submit">Add student</button><div id="notice"
role="status"></div></form></section></div>
<p class="footer">Student Management Portal · Health endpoint: <code>/health</code> · API:
<code>/api/students</code></p></main>
<script>
const byId=id=>document.getElementById(id);
const
esc=value=>String(value).replace(/[&<>"']/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[c])
);
async function refresh(){const [students,health]=await
Promise.all([fetch('/api/students').then(r=>r.json()),fetch('/health').then(r=>r.json())]);byId('count').textCon
tent=students.length;byId('health').textContent=health.status==='ok'?'● Service healthy':'● Service
unavailable';byId('rows').innerHTML=students.map(s=>`<tr><td><b>${esc(s.name)}</b><br><small>${esc(s.email)}</sm
all></td><td class="course">${esc(s.course)}</td><td>${esc(s.year)}</td></tr>`).join('')}
document.getElementById('form').addEventListener('submit',async e=>{e.preventDefault();const
body={name:byId('name').value,email:byId('email').value,course:byId('course').value,year:byId('year').value};con
st r=await
fetch('/api/students',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify(body)});con
st result=await r.json();document.getElementById('notice').textContent=r.ok?`Student ${result.name} registered
successfully.`:result.error;if(r.ok){e.target.reset();refresh()}});
refresh().catch(()=>document.getElementById('health').textContent='● Service unavailable');
</script></body></html>
```

17.4 Dockerfile

```
FROM node:22-alpine
ENV NODE_ENV=production PORT=3000
WORKDIR /app
COPY --chown=node:node package.json server.js index.html students.json ./
EXPOSE 3000
USER node
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 CMD node -e
"fetch('http://127.0.0.1:3000/health').then(r=>process.exit(r.ok?0:1)).catch(()=>process.exit(1))"
CMD ["node", "server.js"]
```

17.5 .dockerignore

```
node_modules
npm-debug.log
.git
.gitignore
Dockerfile
README.md
test
screenshots
```

17.6 Kubernetes Deployment and Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: student-portal
  labels:
    app: student-portal
spec:
  replicas: 2
  selector:
    matchLabels:
      app: student-portal
  template:
    metadata:
      labels:
        app: student-portal
    spec:
      containers:
        - name: student-portal
          image: student-portal:1.0.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 3000
              protocol: TCP
          env:
            - name: PORT
              value: "3000"
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 250m
              memory: 128Mi
          readinessProbe:
            httpGet:
              path: /health
              port: http
            initialDelaySeconds: 3
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: http
            initialDelaySeconds: 10
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: student-portal
spec:
  type: NodePort
  selector:
    app: student-portal
  ports:
    - name: http
      port: 80
      targetPort: http
      protocol: TCP
      nodePort: 30080
```

References

The following primary documentation sources are provided for the technologies and interfaces used in this report.

• **Dockerfile reference** — https://docs.docker.com/reference/dockerfile/

• **Docker build reference** — https://docs.docker.com/build/

• **Kubernetes Deployments** — https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

• **Kubernetes Services** — https://kubernetes.io/docs/concepts/services-networking/service/

- **Kubernetes probes** — https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/
- **Kubernetes resource management** — https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
- **Node.js HTTP API** — https://nodejs.org/docs/latest/api/http.html

Step-by-step visual procedure references

The following visuals are structured walkthrough references for the setup, configuration, execution, and verification steps. They are provided to make the sequence and expected artifacts easier to follow.

Visuals are marked as reference views. Use the actual local command output and application screens when recording your own execution evidence.



Figure IA2-A — Example Dockerfile structure.

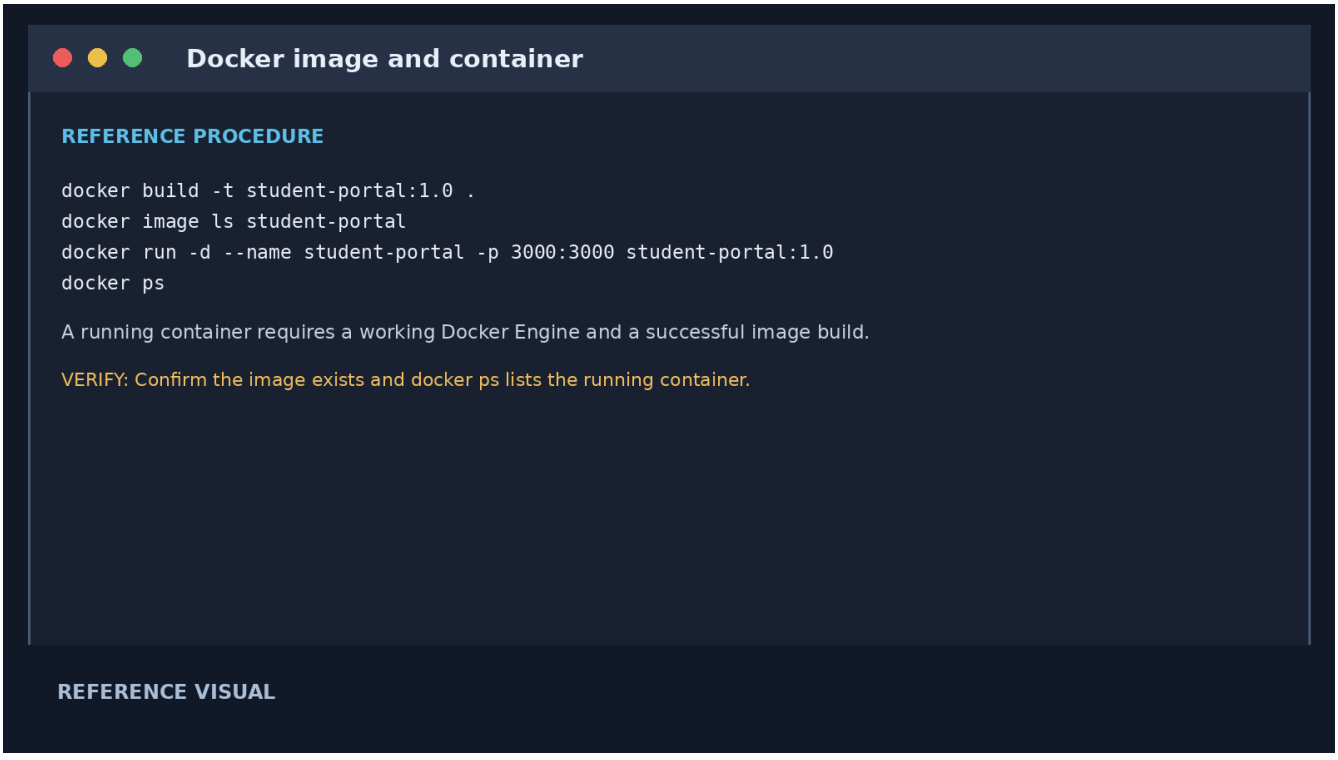


Figure IA2-B — Image build and container verification command sequence.

Kubernetes Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: student-portal
spec:
  replicas: 2
  selector:
    matchLabels:
      app: student-portal
  template:
    metadata:
      labels:
        app: student-portal
    spec:
      containers:
      - name: web
        image: student-portal:1.0
        ports:
```

REFERENCE VISUAL

Figure IA2-C — Example Kubernetes Deployment manifest.

Kubernetes Service verification

REFERENCE PROCEDURE

```
kubectl get deployments,pods,services
kubectl rollout status deployment/student-portal
kubectl get endpoints student-portal
```

A Service selects Pods by labels. Check readiness and endpoints before claiming successful exposure.

VERIFY: Verify two Ready replicas, service endpoints, and access through the Service URL.

REFERENCE VISUAL

Figure IA2-D — Kubernetes rollout, Pod, Service and endpoint verification.

GitHub repository review and external visual references

A public-profile review was performed for the DevOps assignment artifacts. The repository links below are the relevant matches found; the absence of a match means the requested assignment source is not currently visible in the reviewed public repositories.

Profile checked — Profile checked

Public profile search for Docker / Kubernetes / DevOps assignment repositories.

Result — No matching public Docker/Kubernetes assignment repository was located in the repositories reviewed.

No Student Management Portal Dockerfile, Kubernetes Deployment or Service manifest was identified.

Screenshot and documentation references checked

Medium articles were checked for step-by-step visual examples, including Ansible ping/playbook evidence and Jenkins/Docker/Terraform/Kubernetes pipeline views. Official documentation was also checked for accurate command sequences and UI workflows. These sources are learning references, not evidence of execution in this project.

- Ansible official inventory guide
- Docker official container tutorial
- Kubernetes official Pods tutorial
- Jenkins official Pipeline setup guide
- Medium — Ansible deployment with successful ping and playbook examples
- Medium — Jenkins, Docker, Terraform and Kubernetes deployment walkthrough
- Medium — Step-by-step Kubernetes, Terraform and Jenkins guide

Google/search review: web search results were used to locate the Medium articles and official product documentation above. Google is a search index, not the original source of the screenshots; the cited page is the source to consult.